

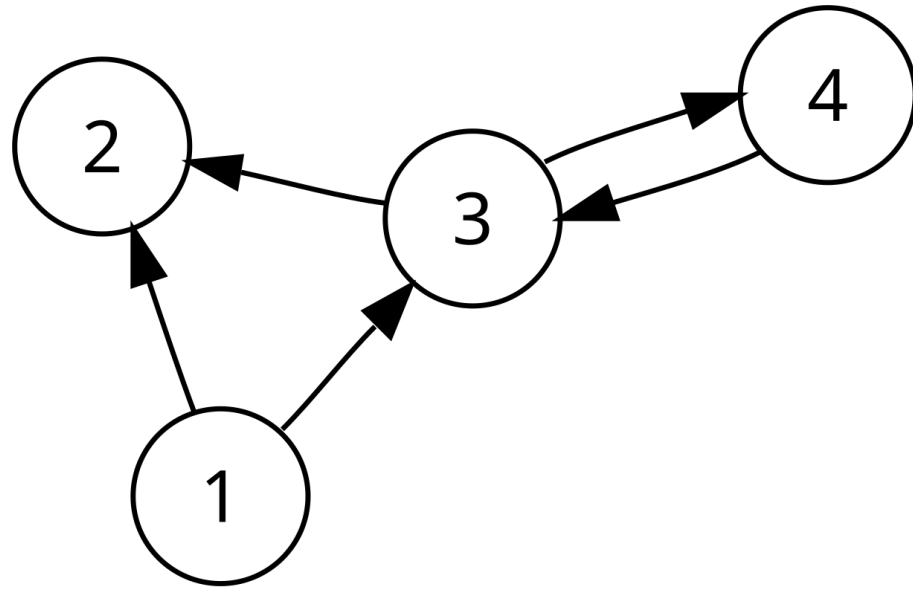
자구&알고 스터디

그래프

BFS, DFS

그래프

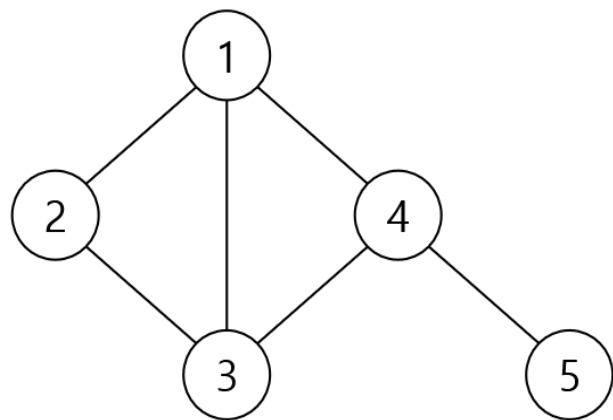
- 유한한 (그리고 아마도 변경 가능한) 정점 집합과 간선(두 정점의 쌍)의 집합으로 이루어진 자료구조를 의미함.
 - 정점을 노드(node)라 하기도 함
 - 간선은 보통 화살로 표현함



그래프 관련 용어

그래프 관련 용어

- 차수(degree): 한 정점에 연결되어 있는 간선의 수
 - 입력 차수(in-degree): 한 정점으로 들어오는 간선의 수
 - 출력 차수(out-degree): 한 정점에서 나가는 간선의 수
- 보행(walk): 정점과 변이 교대로 등장하는 나열, 각 간선의 앞/뒤는 간선이 연결하는 두 정점
 - ex. $1 - \{1,2\} - 2 - \{2,3\} - 3 - \{3,1\} - 1 - \{1,2\} - 2$
- 트레일(trail): 간선이 중복되지 않는 보행
 - ex. $1 - \{1,2\} - 2 - \{2,3\} - 3 - \{3,1\} - 1 - \{1,4\} - 4$
- 경로(path): 정점이 중복되지 않는 트레일
 - ex. $1 - \{1,2\} - 2 - \{2,3\} - 3$
- 사이클(cycle): 시작과 끝점이 동일하고, 나머지는 정점의 중복이 없는 트레일
 - ex. $1 - \{1,2\} - 2 - \{2,3\} - 3 - \{3,1\} - 1$
- 연결 요소(Connected Component): 경로를 통해 서로 연결된 정점들의 집합



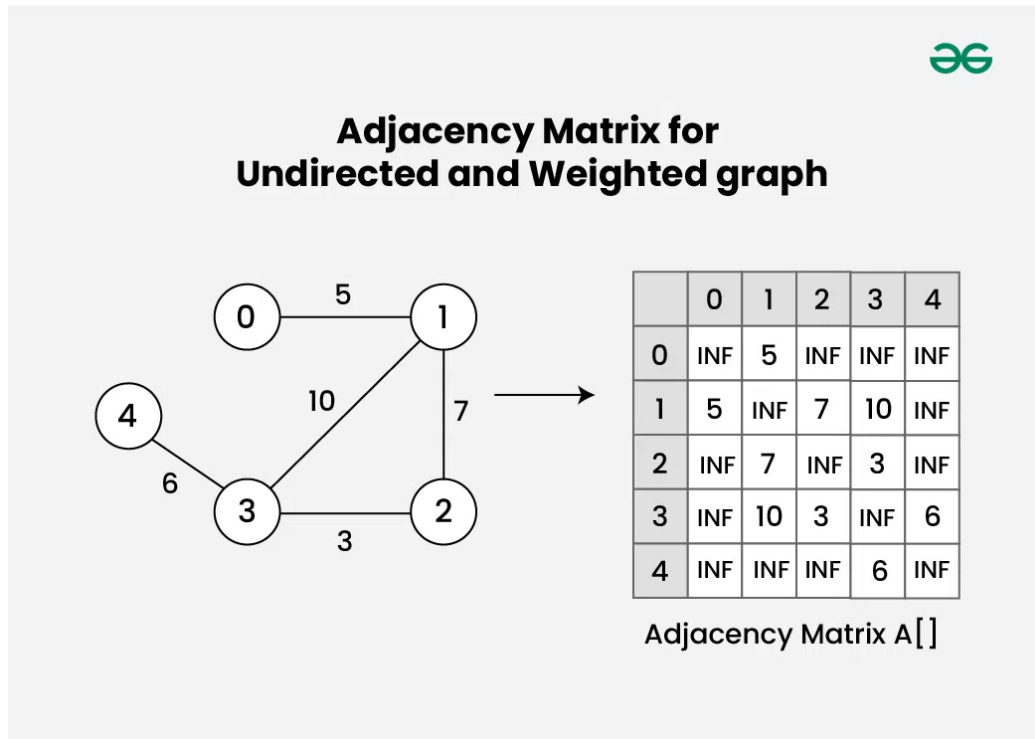
그래프 모양

- 방향 그래프
 - 간선마다 방향이 한 쪽으로 정해진 그래프
- 비방향 그래프
 - 간선마다 방향이 정해져 있지 않는 그래프
 - 방향 그래프의 간선의 방향이 양쪽으로 이동 가능하도록 설정한 것이라 생각하면 편함.
- 가중치 그래프
 - 각 간선마다 어떠한 값이 매겨진 그래프
- 방향 비순환 그래프 (DAG)
 - 방향 그래프이지만 사이클이 없음
- 트리
 - 모든 서로 다른 정점 쌍이 정확히 하나의 경로로 연결된 무방향 그래프 또는 동일하게 연결된 비순환 무방향 그래프

그래프 문제 예시

- 경로 탐색
- 최단 경로
- DP
 - 그래프 노드 색칠

그래프 표현 방법 - 인접 행렬

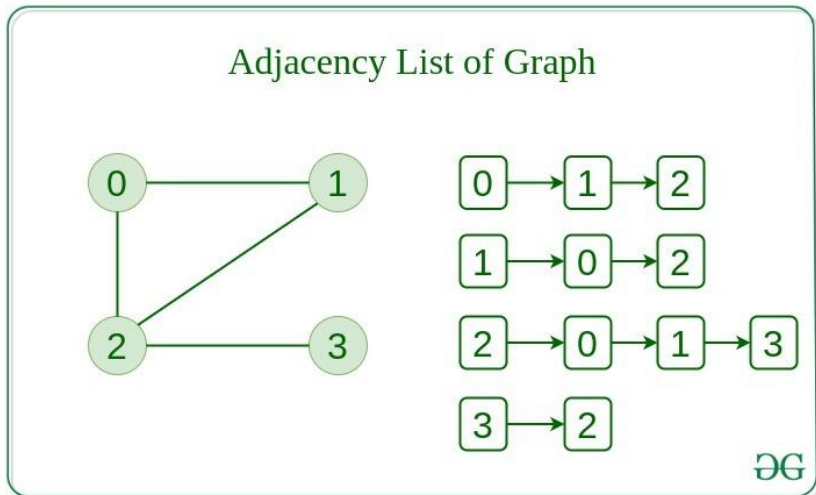


- 간단함
- 만일 방향 그래프이면 행이 간선의 시작점, 열이 간선의 끝점이라 생각하고 구현하면 됨

그래프 표현 방법 - 인접 리스트

```
vector<int> graph[100];  
int main() {  
    int n,m; cin >> n >> m;  
    while (m-- >0) {  
        int s, e; cin >> s >> e;  
        graph[s].push_back(e);  
        // 만일 비방향 그래프인 경우  
        graph[e].push_back(s);  
    }  
}
```

- 각 노드가 어느 노드와 연결 되어 있는지 저장
- 방향 그래프인 경우, 들어오는 노드와 나가는 노드를 따로 기록하는 방법도 있음.
 - 이때, `vector<int> inNode[100];`, `vector<int> outNode[100];` 두 개의 리스트 관리하면 됨.



그래프 순회 - BFS

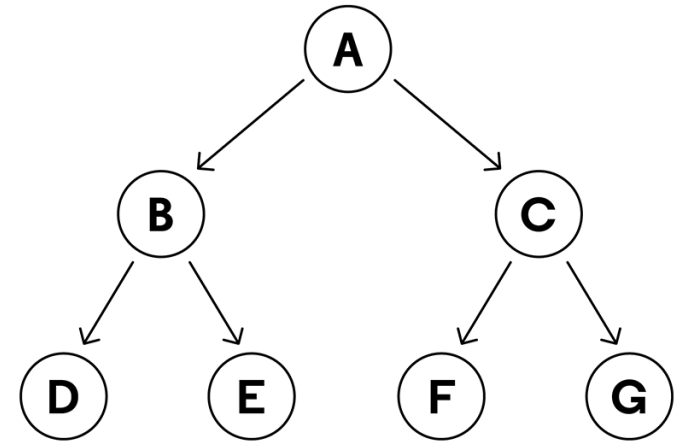
```
int cnt, visited[MAX];  
vector<vector<int>> conn(MAX);
```

```
int main() {  
    /**  
    * BFS  
    * 가까운 위치의 정점부터 탐색하는 알고리즘  
    * 방문할 수 있는 노드가 보이면 큐에 넣는다.  
    */  
    queue<int> q; q.push(r);  
    visited[r] = ++cnt;  
    while(!q.empty()) {  
        int cur = q.front(); q.pop();  
        for(int next: conn[cur]) {  
            if(!visited[next]) { // 방문하지 않은 정점이면 방문  
                visited[next] = ++cnt;  
                q.push(next);  
            }  
        }  
    }  
}
```



Frontier Queue
FIFO (First in First Out)

Tree with an Empty Queue



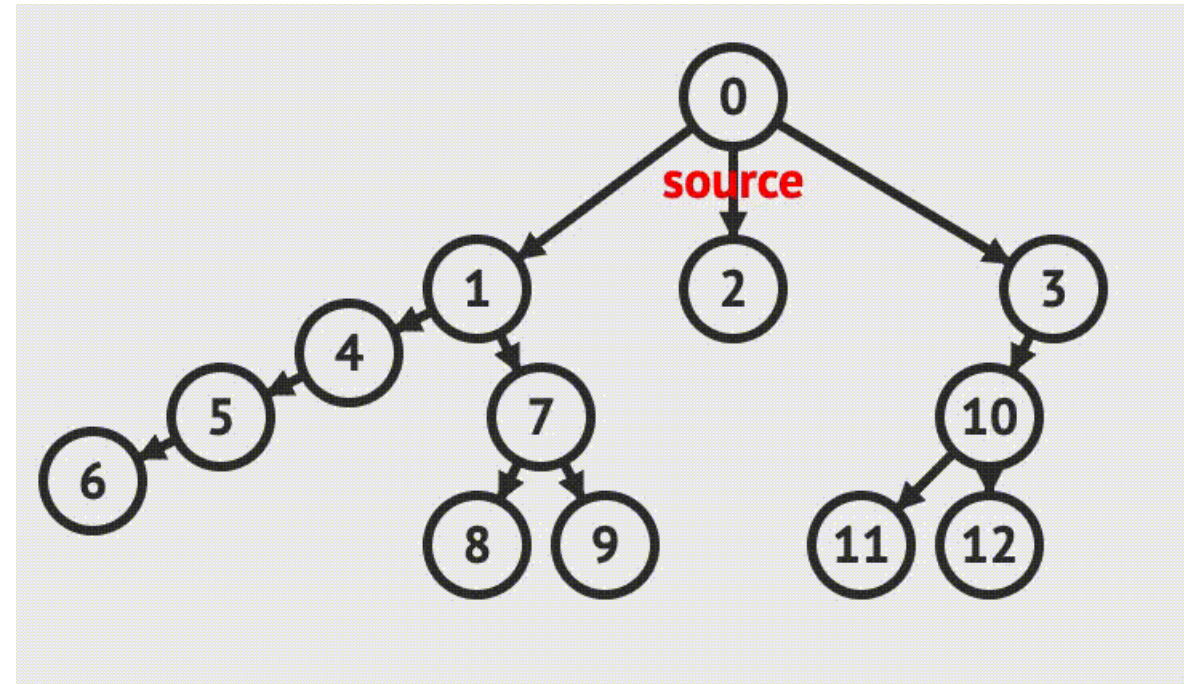
출처: <https://www.codecademy.com/article/bfs-vs-dfs>

출처: 대회 / 코딩테스트 알고리즘 정리, 김서진, <https://github.com/rlatjwls7882>
이 저작물은 CC BY-NC-SA 2.0 KR 라이선스에 따라 이용하였습니다.

그래프 순회 – DFS (재귀)

```
int cnt, visited[MAX];
vector<vector<int>> conn(MAX);

/**
 * DFS
 * 방문할 수 있는 노드가 보이면 이동.
 * 방문할 수 있는 노드가 보이지 않으면 다시 뒤로 되돌아
  감.
 * 미로 찾기에서 왼손의 법칙을 쓰는 것과 같음.
 */
void dfs(int cur) {
    visited[cur] = ++cnt; // 방문 순서 마킹
    for(int next:conn[cur]) {
        if(!visited[next]) { // 방문하지 않은 정점이면 방문
            dfs(next);
        }
    }
}
```

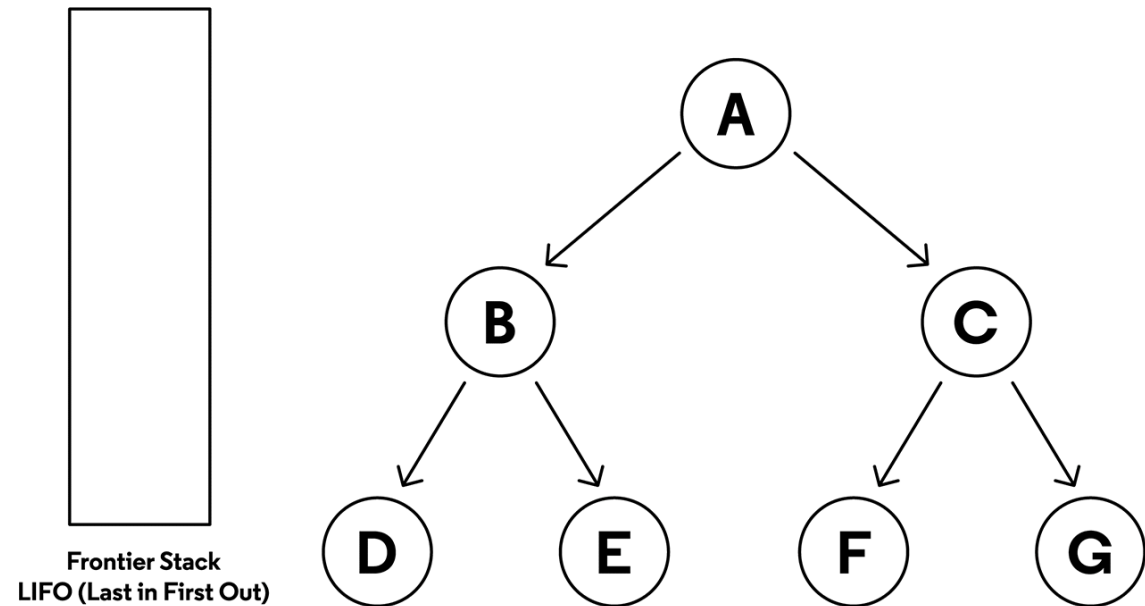


출처:<https://nukw0n-dev.tistory.com/5>

그래프 순회 – DFS (스택)

```
stack<int> q;  
q.push(0);  
vector<bool> visited(graph.size(), false);  
  
while (!q.empty()) {  
    int node = q.top();  
    q.pop();  
  
    if (visited[node])  
        continue;  
  
    visited[node] = true;  
  
    for (int nextNode: graph[node]) {  
        if (visited[nextNode])  
            continue;  
        q.push(nextNode);  
    }  
}
```

Tree with an Empty Stack



출처: <https://www.codecademy.com/article/bfs-vs-dfs>